SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE CODE: CSA15

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Total Marks: 25

Dave's Taxonomy: Manipulation (Level 2) Question
Count: 5

Code of Conduct

I R.YUVASHREE(192521485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:R.YUVASHREE

R**Assessment Tasks**

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
   Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application

Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3.An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues. ● Evaluate the advantages and limitations of serverless computing for this workload.

4.A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources. ● Explain how data quality, metadata management, and governance should be maintained. ● Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5.A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference. ● Discuss the use of cloud-native AI/ML services for model deployment and monitoring. ● Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.
- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

1. CLOUD ARCHITECTURE FOR A VIDEO STREAMING PLATFORM

Scenario

A media company wants to migrate its video streaming platform to the cloud. The system must support millions of concurrent users with minimal buffering and optimized operating costs.

a) Design a cloud architecture using scalability and caching techniques.

Introduction

Cloud computing enables a video streaming company to deliver video content to millions of users without maintaining all the physical servers in its own data centre. The architecture should provide high performance, low latency, scalability, and reliable content delivery.

Proposed Cloud Architecture

The following architecture can be used:

Users → Global DNS → Content Delivery Network (CDN) → Load Balancer → Application Servers → Database and Cloud Storage

Video requests that are not available in the CDN cache are forwarded to the origin storage.

Components of the Architecture

1. Global DNS and Traffic Management

A global DNS service directs users to an appropriate endpoint based on location, availability, and routing policies. This helps users connect to a suitable content delivery location.

2. Content Delivery Network (CDN)

A CDN stores frequently accessed videos at geographically distributed edge locations. When users request a video, the CDN delivers cached content from a nearby location instead of retrieving it from the original server every time.

Benefits of CDN include:

- Reduced video loading time.

- Lower network latency.
- Reduced traffic to origin servers.
- Improved performance during peak usage.

3. Cloud Object Storage

Videos are stored in scalable object storage. The company can store original video files and different versions created through transcoding. Object storage provides durability and allows the platform to store large amounts of video content.

4. Application Servers

Application servers handle user authentication, subscriptions, search, recommendations, and playback requests. Stateless application services make it easier to add or remove server instances when demand changes.

5. Database Services

A managed database stores user profiles, subscription details, video metadata, and viewing history. Frequently accessed information can be cached to reduce repeated database queries.

6. Video Transcoding

Video transcoding converts uploaded videos into different resolutions and bitrates. Adaptive bitrate streaming allows the player to switch video quality according to the viewer's available bandwidth.

7. Monitoring Services

Cloud monitoring tools track response time, bandwidth usage, server utilization, errors, and cache performance. This information helps administrators identify bottlenecks.

Scalability Techniques

Scalability is the ability of a system to handle increasing workloads by allocating additional resources.

- **Horizontal scaling:** Adds more application server instances.
- **Vertical scaling:** Increases the CPU, memory, or capacity of an existing server.
- **Automatic scaling:** Adjusts resources based on workload metrics.
- **Distributed storage:** Supports large video libraries and growing demand
- **Caching:** Reduces repeated requests to databases and origin servers.

Horizontal scaling is particularly useful for a streaming platform because many application instances can process requests simultaneously.

b) Explain how load balancing and auto-scaling ensure smooth streaming.

Load Balancing

Load balancing distributes incoming application requests among multiple servers. It prevents a single server from becoming overloaded when millions of users access the platform.

The load balancer checks server health and forwards requests only to healthy instances. If a server becomes unavailable, traffic can be redirected to other functioning servers.

For example, during a major sports event, a large number of users may start streaming simultaneously. The load balancer distributes their application requests across available servers, improving responsiveness.

Auto-Scaling

Auto-scaling automatically increases or decreases computing resources according to workload demand.

Working process:

1. The cloud monitoring system detects increasing traffic.
2. CPU usage, request rate, or response latency crosses a configured threshold.
3. The auto-scaling service launches additional application instances.
4. The load balancer begins directing requests to the new instances.
5. When demand decreases, unnecessary instances are gradually removed.

Combined Benefits

Load balancing and auto-scaling work together to

- Prevent application server overload.
- Improve application availability.
- Handle sudden traffic spikes
- Reduce response time.
- Maintain performance during peak hours.
- Avoid paying for unnecessary computing resources.

> However, these techniques must be combined with CDN caching and adequate network capacity. Auto-scaling alone cannot eliminate buffering caused by a user's slow internet connection.

c) Discuss cost optimization strategies for high-demand workloads.

High-demand streaming workloads can generate substantial computing, storage, and network expenses. The following strategies help reduce costs.

1. CDN Caching: Frequently watched videos are served from edge locations, reducing origin requests and repeated data transfer from the central storage.

2. Auto-Scaling: Resources are increased during high demand and reduced during low demand.

3. Storage Tiering: Frequently accessed content remains in faster storage, while older or rarely accessed videos can be moved to lower-cost storage tiers.

4. Efficient Video Compression: Suitable codecs and bitrates reduce the amount of data transmitted without unnecessarily reducing video quality.

5. Reserved Capacity: Predictable baseline workloads can use discounted long-term capacity commitments where available.

6. On-Demand Resources: Temporary capacity can handle unexpected demand without maintaining peak infrastructure continuously.

7. Cost Monitoring: Budgets, billing reports, and cost alerts help identify expensive services and unusual usage.

8. Resource Optimization: Unused servers, unattached storage volumes, and unnecessary resources should be removed.

9. Data Transfer Optimization: CDN placement and appropriate routing reduce avoidable network traffic and associated transfer charges.

Conclusion

A combination of cloud storage, CDN caching, load balancing, and auto-scaling enables a video streaming platform to serve millions of concurrent users efficiently. Cost monitoring and resource optimization help maintain performance without unnecessary expenditure.

2. CONTAINER ORCHESTRATION AND MULTI-CLOUD DEPLOYMENT FOR A FINTECH STARTUP

Scenario

A fintech startup deploys containerized applications across multiple cloud providers but faces difficulties maintaining consistent environments and managing deployments.

a) Propose a solution using container orchestration and multi-cloud strategies.

Introduction

Containers package an application together with its dependencies, libraries, and configuration requirements. They help applications run consistently across development, testing, and production environments.

A multi-cloud strategy uses services from two or more cloud providers. Container orchestration helps manage these applications consistently across the different environments.

Proposed Architecture

Developer → Source Code Repository → CI/CD Pipeline → Container Registry → Kubernetes Clusters in Cloud A and Cloud B → Monitoring System

1. Containerization Using Docker

Docker can package application components into portable container images. The same tested image can be deployed across different cloud providers, reducing differences between environments.

For a fintech application, separate containers may be created for:

- User authentication.
- Payment processing.
- Transaction management.
- Customer notifications.
- Fraud detection.

Separating application components makes them easier to deploy, update, and scale independently.

2. Container Orchestration Using Kubernetes

Kubernetes manages containerized applications across a cluster of machines.

Its important functions include:

- Automated container deployment.
- Restarting failed containers.
- Scaling application replicas.
- Service discovery and networking.
- Rolling updates and rollbacks.
- Resource allocation and health monitoring.

Managed Kubernetes services can reduce the operational burden of maintaining the control plane.

3. Container Registry

A container registry stores versioned application images. Access should be restricted, and images should be scanned for vulnerabilities before deployment.

Both cloud environments can retrieve approved versions of the same application image.

4. CI/CD Pipeline

Continuous Integration and Continuous Delivery or Deployment automate software testing and release.

The pipeline should:

- Retrieve source code from version control
- Run automated tests.
- Scan code and container images for vulnerabilities.
- Build and version the container image.
- Publish the approved image to a registry
- Deploy it to a testing environment.
- Promote the same tested artifact to production.
- Support rollback if a deployment fails.

5. Infrastructure as Code

Infrastructure-as-code tools define networks, clusters, access permissions, and other infrastructure through reusable configuration files. They help maintain similar environments across cloud providers.

Provider-specific settings may still be required, but automation reduces configuration differences and manual errors.

b) Explain how service discovery and scaling can be handled efficiently.

Service Discovery

Service discovery allows one application component to locate another component without depending on a fixed IP address.

In Kubernetes, a Service provides a stable endpoint for a group of application pods. Cluster DNS allows applications to locate services using names.

For communication between cloud environments, the startup can use:

- Multi-cluster service discovery.
- A service mesh with cross-cluster support.
- Private network connectivity.
- API gateways for controlled external communication.
- Global traffic management for user-facing endpoints.

For financial applications, internal services should not be exposed publicly without a clear business requirement. Mutual TLS and strict access policies can help protect service-to-service communication.

Scaling Techniques

Horizontal Pod Autoscaling: Adds or removes application replicas according to CPU, memory, or custom application metrics.

Cluster Autoscaling: Adds or removes worker nodes according to resource requirements.

Load Balancing: Distributes requests across healthy application replicas.

Resource Requests and Limits: Specify expected and maximum resource usage to improve scheduling and reduce resource contention.

Health Probes: Readiness probes prevent traffic from reaching an application before it is ready. Liveness probes help detect applications that need restarting.

Queue-Based Scaling: Background workers can scale according to the number of pending payment notifications or other queued tasks.

Scaling payment-processing services requires care. The application must prevent duplicate financial transactions through idempotency controls and reliable transaction handling.

c) Highlight the benefits and risks of a multi-cloud deployment model.

Benefits

1. Reduced Vendor Dependence: The organization is less dependent on a single cloud provider.

2. Improved Resilience: Selected workloads may continue operating if one provider experiences a major outage, provided the application and data recovery arrangements are ready.

3. Flexibility: The startup can select suitable services from different providers.

4. Geographic Coverage: Different regions may improve access for customers in different locations.

5. Business Continuity: A tested secondary environment can support recovery from infrastructure failures.

Risks and Challenges

1. Increased Complexity: Each provider may have different networking, identity, monitoring, and deployment services.

2. Higher Costs: Data transfer between providers and duplicated infrastructure can increase expenses.

3. Security Risks: Misconfigured access policies or inconsistent security controls can expose sensitive financial information.

4. Data Consistency: Keeping transactions and databases synchronized across providers can be difficult.

5. Troubleshooting Difficulties: Distributed failures may require investigation across several platforms.

6. Compliance Requirements: Financial data must be handled according to applicable regulatory and data-protection requirements.

Risk Management Measures

The startup should use centralized monitoring, encryption, consistent access policies, infrastructure automation, vulnerability scanning, and regular recovery testing. It should also evaluate whether multi-cloud complexity is justified by its business requirements.

Conclusion

Docker and Kubernetes help maintain consistent container deployments across multiple environments. Service discovery, autoscaling, CI/CD automation, and infrastructure as code improve reliability and operational efficiency. Multi-cloud deployment provides flexibility and resilience, but requires strong governance, security, and cost management.

3. BACKUP AND DISASTER RECOVERY FOR AN E-LEARNING PLATFORM

Scenario

An e-learning platform loses data due to accidental deletion in the cloud. The organization requires a reliable backup and disaster recovery strategy.

a) Design a solution using cloud-native backup, replication, and versioning.

Introduction

Backup and disaster recovery are essential for protecting data and maintaining service availability. An e-learning platform may store student records, course materials, examination results, assignment submissions, attendance records, and payment information.

Accidental deletion, software errors, cyberattacks, and regional outages can affect these resources. Therefore, the organization should implement multiple layers of protection.

Proposed Architecture

Users and Applications → Primary Cloud Database and Object Storage → Automated Backups and Versioning → Replication to a Separate Region or Account → Recovery Environment

1. Automated Cloud Backups

The organization should configure automatic backups for databases, application files, and important configuration data.

Database backups should support point-in-time recovery where available. This allows the organization to restore the database to a selected point before the deletion or corruption occurred.

Backup schedules and retention periods should be defined according to the importance of the data.

2. Object Storage Versioning

Object storage versioning keeps earlier versions of files when objects are overwritten or deleted.

For example, if an administrator accidentally deletes a course PDF, the platform can recover an earlier version instead of recreating the file.

Versioning is particularly useful for:

- Course videos and documents.
- Student assignment files.
- Learning resources.
- Reports and exported records.

Versioning should be combined with appropriate access controls and retention policies.

3. Cross-Region Replication

Critical data can be replicated to another cloud region. If the primary region becomes unavailable, the organization can use the replicated data in the recovery region.

Replication improves availability, but it is not a complete backup solution. Accidental deletions and corrupted data may also be replicated.

4. Isolated and Immutable Backups

A separate backup account or isolated backup environment reduces the risk that a compromised production account can destroy all recovery copies.

Immutable backups prevent changes or deletion during a defined retention period, according to the chosen storage policy.

5. Encryption and Access Control

Backups should be encrypted at rest and during transfer. Only authorized personnel should be allowed to access, restore, or delete backup data.

Administrative actions should be logged and monitored.

6. Regular Recovery Testing

The organization should periodically restore databases and files in a test environment. This confirms that backups are usable and recovery procedures work correctly.

b) Explain how Recovery Time Objective (RTO) and Recovery Point Objective (RPO) are achieved.

Recovery Time Objective (RTO)

RTO is the maximum acceptable duration for restoring a service after an interruption.

For example, if the platform defines an RTO of two hours, it should aim to restore the required service within two hours of the disruption.

RTO can be reduced through

- Preconfigured recovery infrastructure.
- Automated deployment and restoration.
- Documented recovery procedures.
- Ready-to-use backup environments.
- Regular disaster recovery exercises.

- Automated traffic switching where appropriate.

Recovery Point Objective (RPO)

RPO is the maximum acceptable amount of data loss expressed as a period of time.

For example, an RPO of 15 minutes means the organization aims to recover data with no more than 15 minutes of lost updates.

RPO can be reduced through:

- Frequent database backups.
- Transaction-log backups.
- Point-in-time recovery.
- Continuous or near-real-time replication.
- Regular verification of backup completion.

Difference Between RTO and RPO

RTO focuses on **how quickly the service must be restored**.

RPO focuses on **how much recent data the organization can afford to lose**.

The organization should define separate RTO and RPO targets for critical services, such as student records, examination results, and video content. The backup frequency, replication design, and recovery infrastructure must be selected to meet those targets.

c) Provide steps to ensure business continuity during failures.

Step 1: Identify Critical Services

List essential applications, databases, student records, and learning resources. Determine their recovery priorities.

Step 2: Detect and Assess the Incident

Use monitoring and alerts to identify failures. Determine whether the problem involves accidental deletion, database corruption, a security incident, or a regional outage.

Step 3: Activate the Disaster Recovery Plan

Notify the technical team and relevant administrators. Assign responsibilities and follow the documented recovery procedure.

Step 4: Protect Remaining Data

If corruption or a cyberattack is suspected, isolate affected systems and protect clean backup copies from further damage.

Step 5: Restore Data and Applications

Restore data from the correct backup or version. Use point-in-time recovery when required to avoid restoring the corrupted state.

Step 6: Validate the Recovery

Check database consistency, student records, course files, login functionality, and application performance.

Step 7: Resume Services

Redirect users to the recovered environment where necessary. Inform students and teachers about service availability and alternative arrangements.

Step 8: Monitor and Improve

Monitor the recovered system, document the cause of the incident, and improve backup policies based on the lessons learned.

Conclusion

A reliable disaster recovery strategy combines automated backups, object versioning, cross-region replication, immutable copies, and regular restoration tests. RTO and RPO help the organization measure recovery requirements and select appropriate technologies. These practices reduce data loss, downtime, and disruption to students and teachers.

4. SECURE HYBRID CLOUD FOR A GOVERNMENT AGENCY

Scenario

A government agency wants to adopt a hybrid cloud model for sensitive and non-sensitive information. Some applications must remain on-premises because of compliance requirements.

a) Design a hybrid architecture that ensures secure communication between environments.

Introduction

A hybrid cloud combines on-premises infrastructure with cloud services. It allows an organization to retain sensitive workloads in its own data centre while using cloud resources for approved applications and services.

For a government agency, the architecture must satisfy data classification, access control, confidentiality, integrity, availability, and regulatory requirements.

Proposed Hybrid Cloud Architecture

Government Users → Identity Provider and Access Controls → Agency Data Centre ↔ Secure Private Connection or VPN ↔ Approved Cloud Environment

Firewalls, network segmentation, private endpoints, and centralized monitoring protect communication between the two environments.

1. On-Premises Environment

Sensitive databases and applications that must remain under agency control are hosted in the government data centre.

Examples may include classified records, restricted administrative systems, and applications subject to specific data residency requirements.

Access should be restricted to authorized users and systems.

2. Cloud Environment

Approved cloud services can host less-sensitive applications, public information portals, collaboration services, and scalable public-facing systems.

Before migration, the agency must verify that the selected cloud provider, service, and region meet the relevant government requirements.

3. Secure Network Connectivity

The agency can use a private dedicated connection between its data centre and cloud environment. An encrypted site-to-site VPN may serve as an alternative or backup connection.

Firewalls and network access rules should permit only required communication. Private endpoints can reduce the need to expose internal services to the public internet.

4. Network Segmentation

Separate network zones should be created for sensitive systems, application services, management systems, and public-facing services.

If a public-facing application is compromised, segmentation helps prevent unauthorized access to sensitive databases.

5. API Gateway and Integration Layer

An API gateway or controlled integration service can manage data exchange between cloud and on-premises applications.

It can enforce authentication, authorization, rate limits, and request logging. Only required information should be transferred between environments.

6. Centralized Monitoring

Logs from both environments should be collected in a protected monitoring system. Security alerts should identify unusual access, failed authentication attempts, and suspicious administrative activity.

b) Explain how identity management and encryption should be implemented.

Identity Management

Identity management ensures that only authorized users and services can access government resources.

1. Centralized Identity Provider: A common identity system can manage user accounts and support secure identity federation between environments.

2. Single Sign-On (SSO): SSO allows authorized users to access approved services using a central authentication system.

3. Multi-Factor Authentication (MFA): MFA requires additional verification beyond a password, especially for administrative and sensitive access.

4. Role-Based Access Control (RBAC): Permissions are assigned according to job responsibilities. Users should receive only the access needed for their tasks.

5. Privileged Access Management: Administrator accounts should receive additional protection, restricted access, and enhanced monitoring.

6. Regular Access Reviews: Permissions should be reviewed when employees change roles or leave the organization.

7. Audit Logging: Authentication events, permission changes, and sensitive operations should be recorded and reviewed.

Encryption

Encryption protects data against unauthorized access.

1. Encryption at Rest: Databases, files, disks, and backups should be encrypted using approved cryptographic mechanisms.

2. Encryption in Transit: TLS should protect application communication. VPN or approved private connectivity can secure communication between environments.

3. Key Management: Encryption keys should be stored and managed through approved key-management systems. Access should be restricted, and keys should be rotated according to policy.

4. Secrets Management: Passwords, API keys, certificates, and service credentials should be stored in a secure secrets-management service rather than in application code.

5. Data Classification: Different data categories may require different access, retention, and encryption controls.

6. Secure Deletion and Retention: Information should be retained and disposed of according to legal and government policies.

c) Discuss challenges in managing hybrid cloud environments.

1. Complex Management

On-premises systems and cloud services may use different tools and operating procedures. Standardized automation and documented processes can reduce inconsistency.

2. Security and Compliance

Security policies must be applied consistently across both environments. Regular audits, configuration checks, and centralized logging help identify violations.

3. Network Latency

Communication between on-premises systems and cloud applications may introduce delays. Applications should be placed according to their latency requirements and data dependencies.

4. Data Integration

Different databases and application formats may make information exchange difficult. Standard APIs and controlled integration services can improve interoperability.

5. Cost Management

The agency must manage cloud usage, network connectivity, licensing, and on-premises maintenance costs. Budgets and cost reports help identify unnecessary spending.

6. Skills and Training

Staff must understand cloud administration, networking, identity management, encryption, and compliance requirements. Training and clear responsibilities are essential.

7. Disaster Recovery

Recovery plans must include both on-premises and cloud components. Regular exercises should verify that systems can recover in the correct order.

Conclusion

A secure hybrid cloud architecture allows a government agency to use cloud scalability while retaining sensitive workloads on-premises. Private connectivity, network segmentation, centralized identity management, encryption, and continuous monitoring help protect government information. Strong governance and regular compliance assessments are necessary for successful hybrid cloud operations.

5. MULTI-REGION HIGH AVAILABILITY AND FAULT TOLERANCE FOR A SAAS PROVIDER

Scenario

A Software as a Service (SaaS) provider must ensure that its application continues operating even if one cloud region becomes unavailable.

a) Propose a multi-region deployment strategy in the cloud.

Introduction

High availability means that an application remains accessible with minimal interruption. Fault tolerance is the ability of a system to continue operating despite failures in individual components.

A multi-region architecture deploys application resources across separate geographical cloud regions. It reduces dependence on a single region and improves disaster recovery capability.

Proposed Architecture

Users → Global Traffic Manager → Region A and Region B → Application Servers → Database and Object Storage

Both regions should use multiple availability zones where supported.

1. Primary and Secondary Region

The SaaS provider can deploy the main application in Region A and maintain Region B as a warm standby. The secondary region contains the necessary application components and replicated data.

Alternatively, an active-active architecture allows both regions to serve users simultaneously. This may provide better utilization and resilience but requires more complex application and data design.

2. Global Traffic Management

A global traffic manager directs users to a healthy region using DNS-based routing or another supported global routing mechanism.

Health checks detect unavailable endpoints. When the primary region fails, traffic can be redirected to the secondary region according to the configured failover policy.

DNS caching and client behavior may delay the effect of routing changes, so DNS failover alone does not guarantee an immediate switch.

3. Application Deployment

The same tested application version should be deployed in both regions. Infrastructure-as-code and CI/CD pipelines can maintain consistent infrastructure and configurations.

Application servers should be stateless where possible. Session data should be stored in a shared or replicated service so that users can continue their sessions after traffic moves to another region.

4. Database and Storage

Critical databases should have an appropriate replication and recovery design. Files and backups should also be available in the secondary region.

The architecture must account for replication delays and the possibility that recent updates have not reached the secondary region.

5. Independent Regional Resources

Each region should have sufficient capacity to handle its expected workload during failover. If the secondary region is too small, it may become overloaded when all users are redirected to it.

6. Regular Deployment Testing

The provider should verify that application versions, network rules, access permissions, certificates, and database connections work correctly in both regions.

b) Explain how data replication and failover mechanisms should be configured.

Data Replication

Data replication copies data from one system or region to another.

Synchronous Replication: Data is confirmed in multiple locations before the write is considered complete. This can reduce data loss but may increase latency and depends on the database architecture and network conditions.

Asynchronous Replication: Data is copied after the primary write completes. It can reduce write latency, but some recent updates may be missing if the primary region fails before replication finishes.

The provider should select a replication method based on consistency, latency, RTO, and RPO requirements.

Database Consistency

The provider must define which region can accept writes and how the system handles simultaneous updates.

A controlled promotion process, quorum mechanisms, or fencing can help prevent split-brain conditions, where two regions incorrectly operate as independent primary writers.

Failover Mechanism

Failover transfers application operations from a failed region to a healthy region.

The process includes:

1. Continuously monitor the health of both regions.
2. Detect a regional failure using reliable health checks
3. Determine whether the outage requires failover.

4. Promote the recovery database or activate the secondary environment when appropriate.
5. Redirect traffic to the healthy region.
6. Verify application availability and data consistency.
7. Monitor performance and error rates after recovery.
8. Restore the original region safely and perform a controlled failback when appropriate.

Failover may be automated or require operator approval depending on the risk of false detection and data inconsistency.

Backup and Recovery

Replication should not replace independent backups. Backups protect against accidental deletion, application bugs, and corruption that may spread to replicated systems.

The provider should maintain protected backups, test restoration regularly, and document recovery responsibilities.

Failback

Failback returns operations to the original region after it has recovered.

Before failback, administrators should verify data synchronization, application health, capacity, and network configuration. A controlled procedure helps prevent data loss and unnecessary downtime.

c) Discuss monitoring and alerting strategies for proactive issue detection.

1. Infrastructure Monitoring

Monitor CPU utilization, memory consumption, disk capacity, network throughput, and availability of resources in each region.

These metrics help detect resource exhaustion and infrastructure failures.

2. Application Performance Monitoring

Track:

- Application response time.
- Request throughput.
- Error rates.
- Failed login attempts.
- API availability.
- Database query latency.

- Background job failures.

These metrics help identify application-level problems before they significantly affect customers.

3. Database Monitoring

Monitor database availability, connection counts, replication lag, transaction failures, storage usage, and backup status.

Replication lag is particularly important because it indicates how far the secondary database may be behind the primary database.

4. Centralized Logging

Collect logs from application servers, databases, network services, and security systems in a centralized logging platform.

Logs help administrators investigate failures, identify root causes, and maintain audit records.

5. Distributed Tracing

Distributed tracing follows a request across multiple services. It helps locate delays and failures in complex applications that use microservices.

6. Health Checks and Synthetic Monitoring

Health checks verify whether services are functioning correctly. Synthetic monitoring performs simulated user actions, such as logging in or retrieving application data, from different locations.

This helps identify problems that infrastructure metrics alone may not reveal.

7. Alerting System

Alerts should be configured for

- Regional service failures.
- High response times.
- Increasing error rates.
- Database replication delays.
- Failed backups
- Low storage capacity.
- Security-related incidents.

Alerts should use meaningful thresholds and service-level objectives to avoid excessive notifications.

8. Incident Response

Notifications should reach the responsible technical team through approved channels. Each alert should identify the severity, affected service, and recommended response procedure.

Runbooks should explain how to investigate common failures and when to activate regional failover.

9. Disaster Recovery Drills

The provider should regularly simulate regional failures, test database restoration, verify traffic switching, and measure actual recovery time.

These exercises reveal configuration errors and confirm whether the system meets its availability and recovery objectives.

Conclusion

A multi-region architecture improves SaaS availability by distributing application resources across separate geographical locations. Data replication, controlled failover, independent backups, centralized monitoring, and regular disaster recovery tests help maintain business continuity. The provider should design the system according to clearly defined availability, RTO, RPO, security, and consistency requirements.